NEWDAWN

FOR THE ENGINEERING TEAM · v1.0

Platform Technical Documentation

What it is, what's built, and the full engineering roadmap

Muhammad Qureshi — Naib Imam, Jamia Masjid-e-Nabawi Qureshi Hashmi, G-11/4 Islamabad
AI researcher · author · founder

Live demo: onepiecejourney-crew.netlify.app

1 · Executive Overview

NewDawn (a.k.a. OnePieceJourney) is a people-owned civic operating system — a single platform that fuses the familiar mechanics of Facebook, Uber/inDrive, Reddit, Binance and online banking with decentralized governance, transparent community treasuries, AI assistance (JARVIS), and a problem→solution→vote→fund→execute→verify workflow. The guiding rule: **AI explains, people decide.**

Core hierarchy: **Individual** → **Community/Society** → **City** → **Country** → **Global**. Each level has its own members, rules (set by member vote), public wallet/treasury, proposals, projects and an open ledger. Content is scoped: a global issue reaches everyone; a country issue reaches that country; a city issue that city; a society issue only its members.

2 · The Product Idea (frontend formula)

- **Facebook** — familiar profiles, feed, groups, posts, comments, uploads.
- **Uber / inDrive** — location, nearby activity, request, matching, live status, earn with your skills.
- **Reddit** — up/down voting, threaded discussions that turn into proposals.
- **Binance / online banking** — wallet, balances, transactions, transparent ledgers.
- **SiteDeck (desktop)** — command-center dashboard, live world map, modular panels.
- **JARVIS** — a personalized AI intelligence + action layer on every screen.

Identity: *a familiar social app on mobile; a powerful civic command center on desktop; a personalized JARVIS inside both.*

3 · What is BUILT today (working demo)

Live at onepiecejourney-crew.netlify.app — a fully interactive front-end demo where all data is stored on the user's own device (localStorage), embodying the "your phone is the database" model.

Module	What works in the demo	Status
Onboarding	Name/phone/email → demo OTP → self-custody wallet (address + seed + tokens)	LIVE
Profile & identity	Personal profile, role, skills, ownership shares (NDS)	LIVE
Societies	Join request → admin approval; members list; per-location demo societies	LIVE
Society treasury	Public DAO wallet visible to all members; moves only by vote; open ledger	LIVE
Governance	Proposals, YES/NO/VETO (≥33% veto blocks), tiers, auto fund-release	LIVE
Discussions	Reddit-style threads; upvotes; discussion → proposal	LIVE
Feed	Facebook-style posts with purpose types; problem→Discuss/Ask-JARVIS/Propose/Support	LIVE
Daily brief	Personalized "here's what needs your attention" + JARVIS suggestion	LIVE
Services	Ride (inDrive-style bidding), food, P2P business directory (0% tax)	LIVE
Disaster relief	Alert → relief account → receipts + ground reports on open ledger	LIVE

Science game	Real-world problem quests → points + token bounties + leaderboard	LIVE
World map	Canvas map: communities, creators, disaster zones	LIVE
Markets	Live BTC/ETH/GOLD via public API (real data in the demo UI)	LIVE
Branch panels	Global → Country → City → Society admin views + join approvals	LIVE
JARVIS (web)	Serverless Gemini assistant; anyone can talk to it; key server-side	LIVE
PWA install	Installable to home screen from the site; offline-capable	LIVE

4 · Current architecture (honest)

The demo is a static, client-side application (HTML/CSS/vanilla JS) deployed on Netlify, plus one serverless function that proxies the AI so the API key stays server-side. There is intentionally no central database yet — state lives in the browser. This proves the UX and the "phones-as-database" concept, but it is NOT yet a multi-user networked system.

- Frontend: static HTML/CSS/JS; PWA (manifest + service worker); Facebook-light + ops-dark themes.
- AI: Netlify serverless function → Google Gemini (key in env var). Powers content + assistant.
- Hosting: Netlify (site) + optional Render/Oracle VM for a 24/7 WhatsApp JARVIS bot.
- Data: on-device localStorage (demo). No shared backend, no real blockchain yet.

5 · Target architecture (what the team must build)

To become a real multi-user platform, the following layers are required:

5.1 Frontend (modernize)

- React + TypeScript, Tailwind, accessible component primitives.
- TanStack Query (server state) + Zustand (light client state); Framer Motion (restrained).
- MapLibre for the world map; Recharts for charts; PWA; i18n with RTL Urdu/Arabic.
- Mobile: 5-tab bottom nav (Home, Explore, Create, Wallet, Profile) + floating JARVIS orb.
- Desktop: top nav + collapsible left rail + modular workspace + right-side JARVIS panel.

5.2 Backend & data

- API: REST/GraphQL (Node/NestJS or similar). Auth (JWT), real OTP (Twilio/local SMS), roles & permissions.
- Database: Postgres (users, societies, proposals, votes, projects, ledger, services, discussions).
- Realtime: WebSockets for feed, votes, chat, live map, notifications.
- Storage: object storage (S3/Cloudflare R2) for media; IPFS for public evidence/receipts.
- Search & geo: geospatial queries for nearby societies/services (PostGIS).

5.3 Blockchain layer (Ethereum ecosystem)

- Smart contracts (Solidity): Society DAO (membership, roles), Voting (proposal, quorum, veto), Treasury (multi-sig, milestone releases), Project Registry (budgets, receipts, verification).
- Start on a testnet (Polygon/Base) — free; move to L2 mainnet for production.
- Wallets: WalletConnect/MetaMask; account-abstraction for smooth UX; keys on the user's device.
- Off-chain ↔ on-chain: index events (The Graph); IPFS hashes anchored on-chain for tamper-proof ledgers.

5.4 Decentralized data (the "phones-as-database" vision)

- Local-first storage on device; sync via CRDTs; peer replication so users' devices form the network.
- Public/verifiable data anchored to IPFS + chain; private data encrypted, key held by the user.
- Honest note: true full decentralization is a research-grade effort — recommend a hybrid (server + on-device + chain-anchored proofs) for the first funded version.

5.5 JARVIS (intelligence layer)

- Context service: permission-scoped user context (location, communities, skills, open votes, wallet).
- Actions: explain proposal, compare options, draft proposal, match services, summarize ledger, translate.
- Governance-safe: JARVIS advises only; it never casts votes or moves funds. Every AI output is labeled.
- Auto-update societies: when members vote a rule change, JARVIS drafts + propagates the updated ruleset.

5.6 Mobile app / APK

- Fastest path: ship the PWA as an installable app now (works today from the site).
- For an actual APK / Play Store: wrap the PWA (Capacitor/TWA) or build React Native — funded phase.

6 · Governance & rules engine

Workflow: **Issue** → **Evidence** → **AI analysis** → **Discussion** → **Proposal** → **Vote** → **Fund** → **Execute** → **Verify**. Each society defines its own rules (quorum %, veto threshold, spending limits, admin powers) by member vote; changes are versioned and audit-logged. Global/City/Country admins coordinate but cannot secretly control votes or funds — every admin action is on a public audit log.

7 · Recommended build order

- 1. Design system + app shell (React/TS).
- 2. Auth + onboarding (real OTP, location consent, purpose, nearby societies).
- 3. Home feed + purposeful posts.
- 4. Community profile + join flow + members + public treasury.
- 5. Proposal + discussion + voting + ledger (DB first, chain second).
- 6. Project tracking + receipts + verification.
- 7. JARVIS context service + actions.
- 8. World map + tiered content scope.
- 9. Marketplace (services, earn-with-skills).
- 10. Emergency module. 11. Science Lab. 12. Admin panels (society→global).
- 13. Smart contracts on testnet → integrate. 14. Harden, audit, pilot.

8 · Status legend & honest scope

Tier	Meaning	Examples
LIVE	Working in the demo today (client-side)	signup, profiles, societies, feed, voting, demo wallet, JARVIS, map, services, brief
PILOT	Next, with a small funded team	real DB + auth, provider marketplace, AI policy analysis, KYC, community fund
ROADMAP	Funded multi-year build	on-chain treasury, real crypto/fiat, cross-border, full ride/delivery ops, 3D science game, gov integration

This honesty is a feature, not a weakness: it makes the platform investable and protects the founder's credibility. Nothing in the demo is presented as a finished, production, or at-scale system.

9 · Team to hire

- Frontend engineers (React/TS, maps, PWA), Backend engineers (Node, Postgres, realtime),
- Blockchain engineers (Solidity, L2, account abstraction), Mobile (RN/Capacitor),
- AI engineer (LLM integration, RAG, safety), DevOps/security, Product designer, and
- domain advisors (governance, Islamic finance/ethics, community pilots).